

Proyecto 2

Big Data & Data Lake en AWS

ST0263 – Tópicos Especiales en Telemática

Universidad EAFIT

2026-1

Integrantes

Valentina Castro Pineda
Juan Pablo Avendaño Bustamante
Diego Andrés González Graciano

Medellín, Colombia

Índice

1. Información del proyecto	2
2. Requerimientos del proyecto	2
3. Desarrollo	3
3.1. Punto 1 — Caso de estudio y preguntas de negocio	3
3.2. Punto 2 — Fuentes de datos (RDS, EC2, URL)	4
3.3. Punto 3 — Ingesta automática al Data Lake	5
3.4. Punto 4 — ETL <code>raw</code> → <code>trusted</code> con AWS Glue + Spark	6
3.5. Punto 5 — Glue Crawler + Data Catalog	7
3.6. Punto 6 — Consultas analíticas (Athena + SparkSQL)	9
3.7. Punto 7 — Análisis con PySpark (zona <code>curated/</code>)	10
3.8. Punto 8 — Dashboard Streamlit + API Gateway	13
4. Video de sustentación	15

1. Información del proyecto

Caso de estudio: Análisis financiero de películas usando múltiples fuentes de datos. Se construye un pipeline de Big Data sobre AWS que ingesta datos desde tres fuentes (archivos en EC2, RDS MariaDB y una URL pública), los procesa con Glue + Apache Spark, los cataloga con Glue Data Catalog, los consulta con Athena y SparkSQL, y los expone en un dashboard Streamlit publicado vía API Gateway.

Enlaces del proyecto

Recurso	URL
Repositorio GitHub	https://github.com/EAFIT-Works/ST0263-261-proyecto2
Video de sustentación	Ver en Google Drive
Entrega al profesor	aeospinas@eafit.edu.co (OneDrive EAFIT)

2. Requerimientos del proyecto

Enunciado

1. Crear un caso de estudio donde procese datos almacenados en fuentes de datos: Bases de Datos, Archivos y URL. Plantear al menos 5 preguntas de negocio sobre esos datos.
2. Fuentes de datos: simular datos en RDS MariaDB, archivos en una instancia de EC2 y URLs en Internet.
3. Ingestar datos hacia el Datalake (AWS S3) en las zonas correctas. Deberá ser un proceso automático.
4. Realizar procesamiento orientado a la Preparación de datos (desde zona raw a trusted) con AWS Glue y Apache Spark.
5. Realizar catalogación de tablas SQL con AWS Glue y EMR/Hive.
6. Realizar consultas y procesamiento analítico descriptivo con SQL utilizando AWS Athena, EMR/Hive y SparkSQL.
7. Realizar consultas y procesamiento analítico descriptivo con Spark en Python (PySpark).
8. Realizar alguna aplicación de visualización de datos y una aplicación expuesta mediante API Gateway con Streamlit de Python.

Preguntas de negocio

- P1.** ¿Qué géneros generan el mayor revenue total?
- P2.** ¿Qué actores aparecen con mayor frecuencia en películas rentables?
- P3.** ¿Cómo evoluciona el revenue (USD y COP) a lo largo de los años?
- P4.** ¿Qué directores tienen el mejor ROI promedio?
- P5.** ¿Existe correlación entre los ratings y las métricas financieras (revenue, profit, ROI)?

3. Desarrollo

3.1. Punto 1 — Caso de estudio y preguntas de negocio

Objetivo

Definir un caso de estudio sobre análisis financiero de películas y formular las 5 preguntas de negocio que orientan todo el pipeline.

La industria del entretenimiento reporta sus métricas financieras (presupuesto, ingresos, ganancia, ROI) en USD, mientras los inversionistas y el público latinoamericanos razonan en COP. El proyecto combina un catálogo de títulos (películas y series), datos transaccionales de personas y reseñas, y tasas de cambio en línea para responder preguntas sobre la rentabilidad histórica.

Distribución entre las 3 fuentes:

Fuente	Tipo	Contenido
EC2 (archivos)	/home/ubuntu/project/data/	movies.csv, tv_shows.csv
RDS MariaDB movies	BD relacional	people, movie_reviews
URL pública	HTTPS / REST	exchangerate-api.com (USD base)

Esquema principal (tabla movies en trusted/): tmdb_id, title, release_year, revenue_usd, revenue_cop, profit_usd, roi_pct, budget_usd, vote_average, cast_ids, directors, genres.

Convenciones del proyecto:

- Bucket S3: movie-analytics-lake2 (región us-east-1).
- Zonas del Data Lake: raw/ (CSV), trusted/ (Parquet limpio), curated/ (Parquet agregado).
- Base Glue Data Catalog: movie_trusted_db.
- Rol IAM único: LabRole (AWS Academy Learner Lab).

3.2. Punto 2 — Fuentes de datos (RDS, EC2, URL)

Objetivo

Configurar tres fuentes de datos heterogéneas (RDS MariaDB, EC2 con archivos y URL pública) que alimentan el Data Lake.

Recurso	Configuración
RDS MariaDB	db.t3.micro, 20 GB gp2, puerto 3306, BD movies
EC2 (Ubuntu 22.04)	t2.micro, IAM Profile LabRole, datos en /home/ubuntu/project/data/
API pública	GET https://api.exchangerate-api.com/v4/latest/USD (sin auth)

Carga de la base de datos: desde el EC2 se ejecuta `punto2/scripts/load_db.data.py`, que lee `people.csv` y `movie_reviews.csv` desde el disco y los inserta en MariaDB por chunks usando SQLAlchemy + Pandas.

```

1  def load_csv(file_name, table_name, chunksize=CSV_CHUNK_SIZE):
2      path = os.path.join(LOCAL_DATA_PATH, file_name)
3      first = True
4      for chunk in pd.read_csv(path, chunksize=chunksize, low_memory=True):
5          chunk.to_sql(
6              name=table_name, con=engine,
7              if_exists="replace" if first else "append",
8              index=False, method="multi",
9          )
10         first = False
11         gc.collect()

```

Listing 1: Carga por chunks a MariaDB.

Cómo ejecutar:

1. Crear la RDS MariaDB y la EC2 con LabRole; abrir SG en 3306 (RDS) y 22 (EC2).
2. Subir los CSV al EC2 con `scp` a `/home/ubuntu/project/data/`.
3. Desde el EC2: `cd ~/project/punto2/scripts && python load_db.data.py`.
4. Validar la URL: `curl -s https://api.exchangerate-api.com/v4/latest/USD`.

```

MariaDB [movies]>
MariaDB [movies]> SHOW TABLES;
+-----+
| Tables_in_movies |
+-----+
| movie_reviews     |
| people            |
+-----+
2 rows in set (0.001 sec)

MariaDB [movies]>

```

Figura 1: Tablas `people` y `movie_reviews` en RDS MariaDB.

3.3. Punto 3 — Ingesta automática al Data Lake

Objetivo

Ingestar las tres fuentes hacia `s3://.../raw/` de forma automática y programada, manteniendo histórico con timestamps.

Tres scripts independientes en `punto3/scripts/` consolidan los datos de las tres fuentes y los depositan en `s3://movie-analytics-lake2/raw/` con timestamp en el nombre del archivo (`YYYYMMDD_HHMMSS`). Un orquestador (`ingest_all.py`) los ejecuta en cadena.

Script	Origen	Destino S3
<code>ingest_ec2.py</code>	EC2 (CSV)	<code>raw/movies/</code> , <code>raw/tv_shows/</code>
<code>ingest_db.py</code>	RDS (<code>people</code> , <code>movie_reviews</code>)	<code>raw/people/</code> , <code>raw/movie_reviews/</code>
<code>ingest_api.py</code>	API exchange-rates	<code>raw/exchange/</code>

```

1 import boto3
2 from datetime import datetime
3
4 s3 = boto3.client("s3")
5 ts = datetime.utcnow().strftime("%Y%m%d_%H%M%S")
6 key = f"raw/{dataset}/{dataset}_{ts}.csv"
7 s3.upload_file(local_path, S3_BUCKET, key)

```

Listing 2: Subida a S3 con timestamp.

Cómo ejecutar:

1. Crear bucket y zonas: `aws s3 mb s3://movie-analytics-lake2` y `aws s3api put-object --key raw/` (igual para `trusted/` y `curated/`).
2. Ejecutar manualmente: `cd ~/project/punto3/scripts && python ingest_all.py`.
3. Programar con cron cada hora: `0 * * * * /home/ubuntu/project/.venv/bin/python /home/ubuntu/project`

Nota: La EC2 usa el Instance Profile `LabRole` (acceso completo a S3), por lo que no se necesitan credenciales explícitas en código.

3.4. Punto 4 — ETL raw → trusted con AWS Glue + Spark**Objetivo**

Transformar datos crudos de `raw/` a `trusted/` utilizando AWS Glue y Apache Spark, produciendo Parquet limpio y tipado, con métricas financieras pre-calculadas.

Glue Job `movie-raw-to-trusted-etl` que lee los CSV de `raw/`, los limpia y los escribe como Parquet en `trusted/`. Produce 6 tablas: `movies`, `movie_genres`, `tv_shows`, `people`, `movie_reviews`, `exchange`.

Transformaciones clave: casting estricto de tipos, normalización de strings nulos (`,` `"null"`, `"N/A"`), deduplicación por clave usando `Window.row_number()`, normalización de géneros, descarte de `tmdb_id` inválidos, `rating` acotado a `[0, 10]`, y — la pieza más importante — recálculo de las métricas financieras desde columnas limpias y conversión a COP usando la tasa de `exchange`:

```

1  movies = movies.withColumn(
2      "profit_usd",
3      when(col("revenue_usd").isNotNull() & col("budget_usd").isNotNull(),
4          col("revenue_usd") - col("budget_usd")),
5  ).withColumn(
6      "roi_pct",
7      when(col("budget_usd").isNotNull() & (col("budget_usd") > 0),
8          (col("revenue_usd") / col("budget_usd")) * 100),
9  )
10
11  cop = exchange.filter(col("currency") == "COP") \
12      .select(col("_rate").alias("_rate")).limit(1)
13  movies = movies.crossJoin(broadcast(cop)) \
14      .withColumn("revenue_cop", col("revenue_usd") * col("_rate"
15      )) \
16      .drop("_rate")

```

Listing 3: Métricas financieras y conversión a COP.

Cómo ejecutar:

1. Subir punto4/scripts/glue_raw_to_trusted.py como script del job o pegarlo en el editor.
2. Configurar: tipo *Spark*, Glue 5.1, IAM role *LabRole*, worker *G.1X*, 10 workers.
3. Job parameters: `--RAW_PREFIX s3://movie-analytics-lake2/raw/`, `--TRUSTED_PREFIX s3://movie-analyt`
4. *Run job*. Tiempo típico: ~2 minutos, ~0.54 DPU.h.

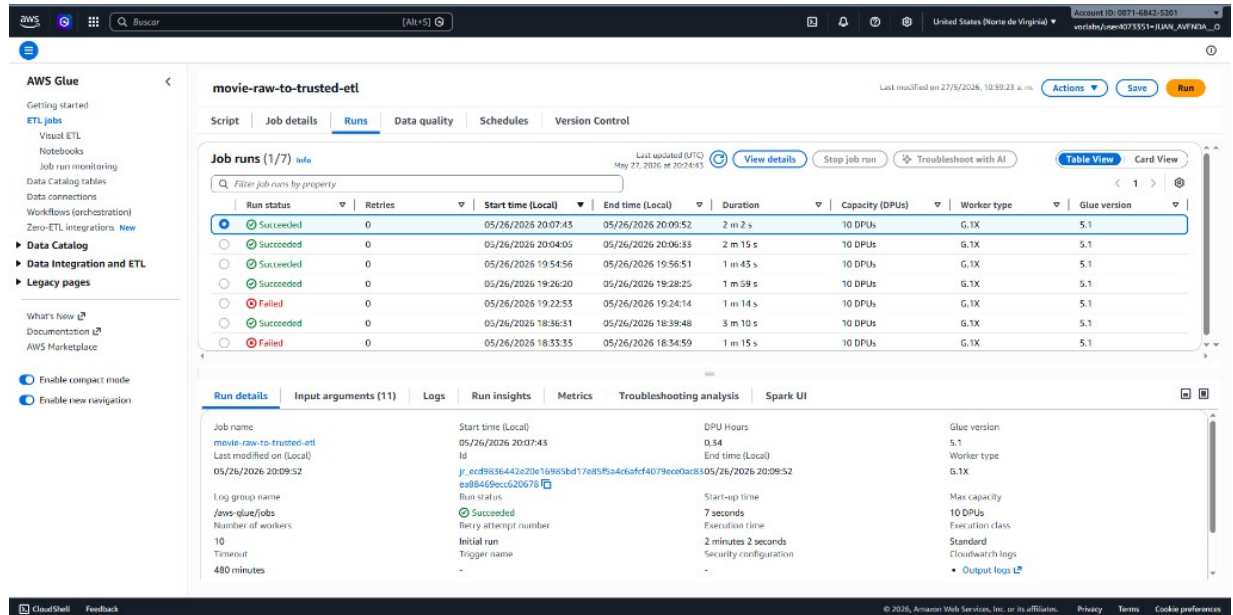


Figura 2: Run exitoso del Glue Job movie-raw-to-trusted-etl.

3.5. Punto 5 — Glue Crawler + Data Catalog

Objetivo

Catalogar las tablas Parquet de **trusted/** en el Glue Data Catalog para hacerlas consultables desde Athena y SparkSQL sin declarar esquemas manualmente.

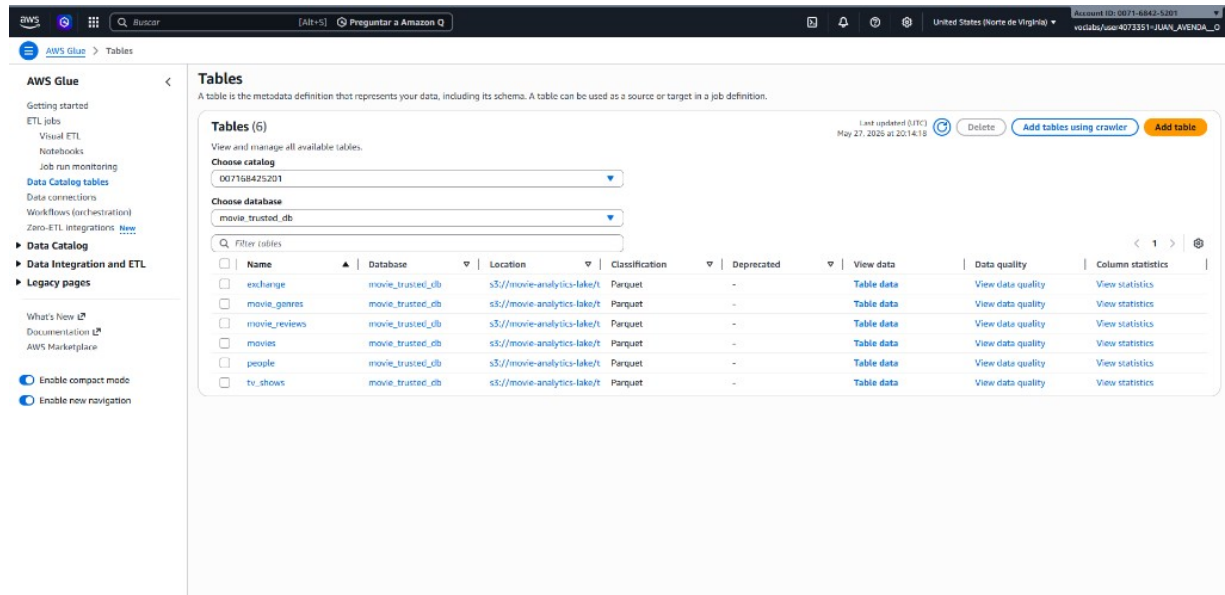
El Crawler **trusted-movies-crawler** recorre `s3://movie-analytics-lake2/trusted/` y crea automáticamente una tabla por carpeta en la database `movie_trusted_db`, infiriendo el esquema directamente de los archivos Parquet (incluyendo columnas derivadas como `profit_usd`, `roi_pct` y `revenue_cop`).

Propiedad	Valor
Nombre	trusted-movies-crawler
IAM role	LabRole
Database destino	movie_trusted_db
Fuente	s3://movie-analytics-lake2/trusted/ (recursivo)
Schedule	On-demand (manual)
Estado	READY

Catalogó las 6 tablas (`movies`, `movie_genres`, `tv_shows`, `people`, `movie_reviews`, `exchange`), todas con `Classification = Parquet`. Estas son las tablas que consumen Athena y SparkSQL en el Punto 6.

Cómo ejecutar:

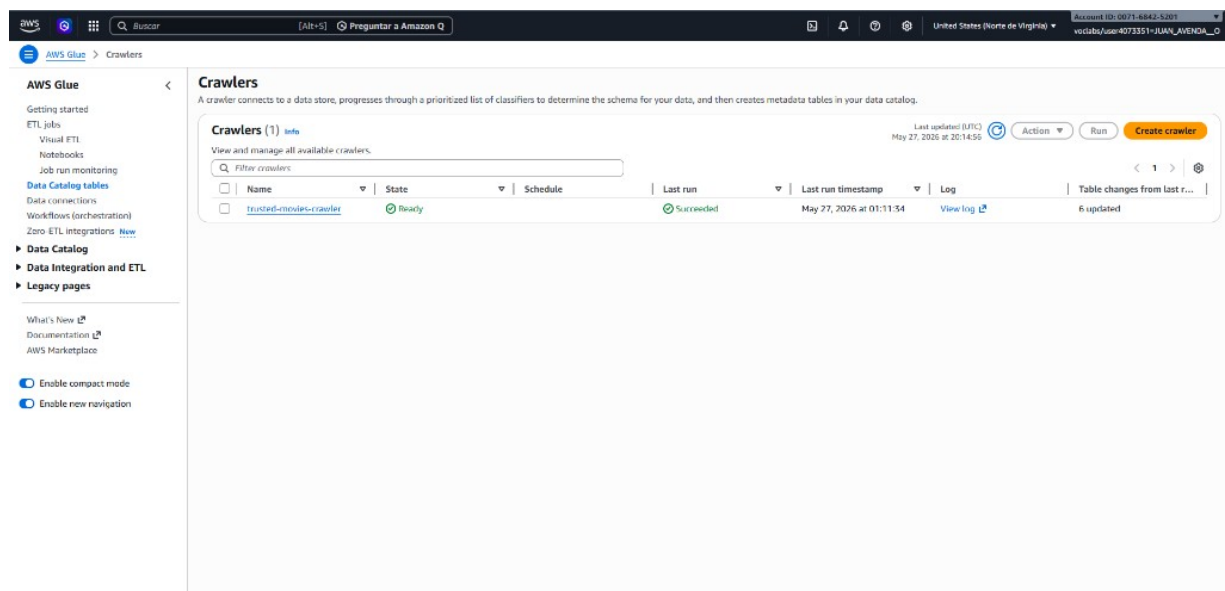
1. Glue → *Crawlers* → *Create crawler* con los valores de la tabla anterior.
2. Crear la database `movie_trusted_db` en el Data Catalog si no existe.
3. *Run crawler*. Duración típica: ~45 s.



The screenshot shows the AWS Glue console 'Tables' page. It displays a list of 6 tables in the 'movie_trusted_db' database, all with a 'Parquet' classification. The tables are: exchange, movie_genres, movie_reviews, movies, people, and tv_shows. Each table has a 'View data' link and a 'View statistics' link. The interface includes a sidebar with navigation options like 'Getting started', 'ETL jobs', 'Visual ETL', 'Notebooks', 'Job run monitoring', 'Data Catalog tables', 'Data connections', 'Workflows (orchestration)', and 'Zero-ETL integrations'. The main content area shows the 'Tables (6)' section with a search bar and a table listing the cataloged tables.

Name	Database	Location	Classification	Deprecated	View data	Data quality	Column statistics
exchange	movie_trusted_db	s3://movie-analytics-lake/t	Parquet	-	Table data	View data quality	View statistics
movie_genres	movie_trusted_db	s3://movie-analytics-lake/t	Parquet	-	Table data	View data quality	View statistics
movie_reviews	movie_trusted_db	s3://movie-analytics-lake/t	Parquet	-	Table data	View data quality	View statistics
movies	movie_trusted_db	s3://movie-analytics-lake/t	Parquet	-	Table data	View data quality	View statistics
people	movie_trusted_db	s3://movie-analytics-lake/t	Parquet	-	Table data	View data quality	View statistics
tv_shows	movie_trusted_db	s3://movie-analytics-lake/t	Parquet	-	Table data	View data quality	View statistics

Figura 3: Las 6 tablas Parquet catalogadas en `movie_trusted_db`.



The screenshot shows the AWS Glue console 'Crawlers' page. It displays a single crawler named 'trusted-movies-crawler' in a 'Ready' state. The crawler was last updated on May 27, 2026, at 20:14:55. The interface includes a sidebar with navigation options like 'Getting started', 'ETL jobs', 'Visual ETL', 'Notebooks', 'Job run monitoring', 'Data Catalog tables', 'Data connections', 'Workflows (orchestration)', and 'Zero-ETL integrations'. The main content area shows the 'Crawlers (1)' section with a search bar and a table listing the crawler.

Name	State	Schedule	Last run	Last run timestamp	Log	Table changes from last r...
trusted-movies-crawler	Ready		Succeeded	May 27, 2026 at 01:11:34	View log	6 updated

Figura 4: Crawler `trusted-movies-crawler` en estado Ready.

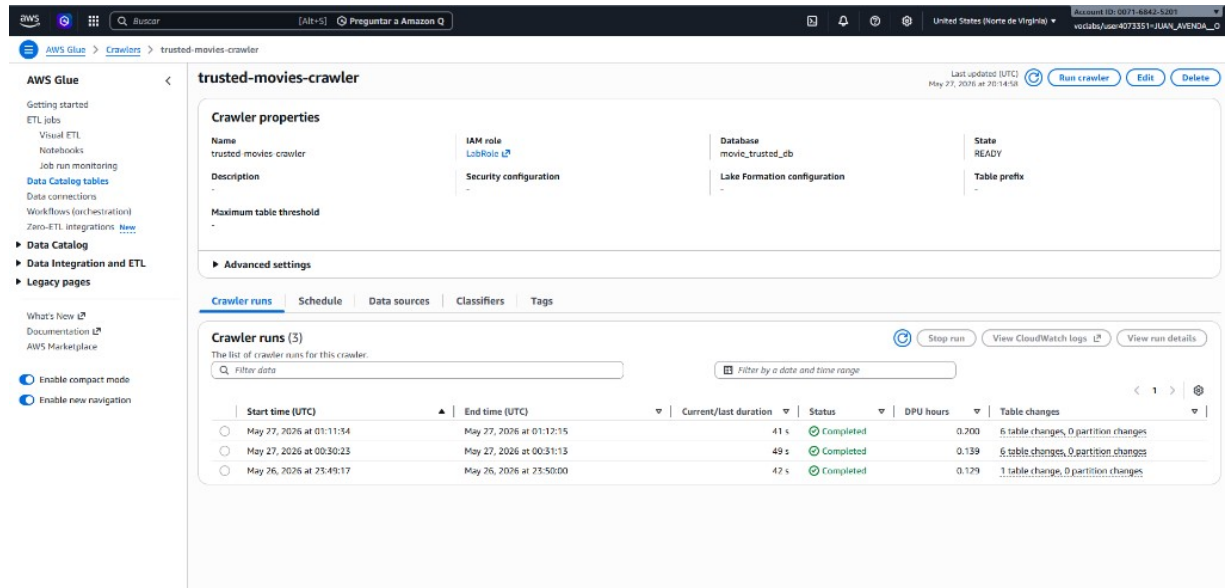


Figura 5: Detalle y runs del crawler con duraciones y *table changes*.

3.6. Punto 6 — Consultas analíticas (Athena + SparkSQL)

Objetivo

Resolver las 5 preguntas de negocio con SQL en dos motores (Athena y SparkSQL) que apuntan a las mismas tablas Parquet catalogadas.

- **Athena:** SQL serverless directamente sobre los Parquet catalogados (`movie_trusted_db`).
- **SparkSQL:** el mismo SQL ejecutado dentro de un notebook PySpark (`punto6/queries/sparksql.queries.ipynb`) sobre vistas temporales registradas desde S3.

```

1 SELECT g.genre,
2        COUNT(DISTINCT g.tmdb_id) AS num_peliculas,
3        ROUND(SUM(m.revenue_usd), 2) AS total_revenue_usd,
4        ROUND(SUM(m.revenue_cop), 2) AS total_revenue_cop
5 FROM movie_genres g
6 JOIN movies m ON g.tmdb_id = m.tmdb_id
7 WHERE m.revenue_usd IS NOT NULL
8 GROUP BY g.genre
9 ORDER BY total_revenue_usd DESC
10 LIMIT 20;

```

Listing 4: P1 — Géneros más rentables.

```

1 WITH review_agg AS (
2     SELECT tmdb_id, AVG(rating) AS avg_review_rating
3     FROM movie_reviews
4     WHERE rating IS NOT NULL

```

```

5      GROUP BY tmdb_id
6      HAVING COUNT(*) >= 3
7  )
8  SELECT CORR(m.vote_average, m.revenue_usd) AS corr_nota_revenue,
9         CORR(m.vote_average, m.profit_usd) AS corr_nota_profit,
10        CORR(m.vote_average, m.roi_pct) AS corr_nota_roi,
11        CORR(r.avg_review_rating, m.revenue_usd) AS corr_resenas_revenue
12 FROM movies m
13 JOIN review_agg r ON m.tmdb_id = r.tmdb_id
14 WHERE m.vote_average IS NOT NULL
15 AND m.revenue_usd IS NOT NULL;

```

Listing 5: P5 — Correlación rating vs ganancias.

Cómo ejecutar:

1. **Athena:** abrir Athena → database movie_trusted_db → ejecutar cada bloque SELECT de punto6/queries/ de uno en uno.
2. **SparkSQL:** abrir el notebook punto6/queries/sparksql_queries.ipynb en Colab o Glue Notebooks, configurar las credenciales temporales del lab, ejecutar todas las celdas.

3.7. Punto 7 — Análisis con PySpark (zona curated/)**Objetivo**

Materializar los resultados de las 5 preguntas en 6 tablas Parquet dentro de la zona **curated/** del Data Lake usando un Glue Job PySpark.

Glue Job movie-trusted-to-curated (punto7/scripts/analisis_pyspark.py) que produce 6 tablas dentro de s3://movie-analytics-lake2/curated/. Estas tablas son la fuente directa del dashboard del Punto 8.

Tabla curated/	Pregunta	Filas
revenue_by_genre	P1: géneros más rentables	~20
top_actors_rentables	P2: actores en películas rentables	25
revenue_by_year	P3: revenue por año (desde 1980)	~50
roi_by_director	P4: directores con mejor ROI	20
ratings_correlations	P5a: 6 coeficientes globales	1
ratings_vs_revenue_sample	P5b: scatter rating vs revenue	~2.000

```

1  by_director = (
2      movies.where(F.col("directors").isNull() & F.col("roi_pct").
3          isNull())
4          .withColumn("director_raw", F.explode(F.split(F.col("directors")
5              , ",")))
6          .withColumn("director_name", F.trim(F.col("director_raw")))
7          .where(F.col("director_name") != "")
8  )

```

```

8 result = (
9     by_director.groupBy("director_name")
10    .agg(
11        F.countDistinct("tmdb_id").alias("n_movies"),
12        F.round(F.avg("roi_pct"), 2).alias("avg_roi_pct"),
13        F.round(F.avg("profit_usd"), 2).alias("avg_profit_usd"),
14    )
15    .where(F.col("n_movies") >= MIN_DIR_MOVIES)
16    .orderBy(F.desc("avg_roi_pct"))
17    .limit(20)
18 )
19 result.write.mode("overwrite").parquet(f"{CURATED}roi_by_director/")

```

Listing 6: P4 — ROI por director.

Cómo ejecutar:

1. Crear el Glue Job `movie-trusted-to-curated` (Spark Script Editor, Glue 5.1, LabRole, G.1X ×5).
2. Job parameters: `--TRUSTED_PREFIX s3://movie-analytics-lake2/trusted/, --CURATED_PREFIX s3://movie-analytics-lake2/curated/, --DECADE_FROM 1980, --MIN_DIR_MOVIES 3, --SAMPLE_SIZE 2000`.
3. *Run job*. Duración obtenida durante nuestra ejecución: **1m 37s, 0.27 DPU·h**.
4. Verificar: `aws s3 ls s3://movie-analytics-lake2/curated/` debe listar los 6 prefijos.

The screenshot displays the AWS Glue console interface. At the top, the job name 'movie-trusted-to-curated' is shown with a 'Run' button. Below this, the 'Job runs (1/1)' section shows a single successful run. The table below provides detailed metrics for the job run.

Run status	Retries	Start time (Local)	End time (Local)	Duration	Capacity (DPUs)	Worker type	Glue version
Succeeded	0	05/27/2026 21:25:10	05/27/2026 21:25:00	1 m 37 s	10 DPUs	G.1X	5.1

The 'Run details' section at the bottom provides further information:

- Job name:** movie-trusted-to-curated
- Last modified (on Local):** 05/27/2026 21:25:00
- Log group name:** /aws-glue/jobs
- Number of workers:** 10
- Timeout:** 60 minutes
- Usage profile:** -
- Start time (Local):** 05/27/2026 21:25:10
- Id:** j_06fe477b54602cd9c4c9c95b12516248ba223a581382bb9dc817a4
- Run status:** Succeeded
- Run status:** Succeeded
- Retry attempt number:** -
- Initial run:** -
- Trigger name:** -
- Job run queuing:** False
- DPU Hours:** 0.271389
- End time (Local):** 05/27/2026 21:25:00
- Start-up time:** 12 seconds
- Execution time:** 1 minute 37 seconds
- Security configuration:** -
- Glue version:** 5.1
- Worker type:** G.1X
- Max capacity:** 10 DPUs
- Execution class:** Standard
- Cloudwatch logs:**
 - Output logs
 - Error logs

Figura 6: Run exitoso del Glue Job `movie-trusted-to-curated`.

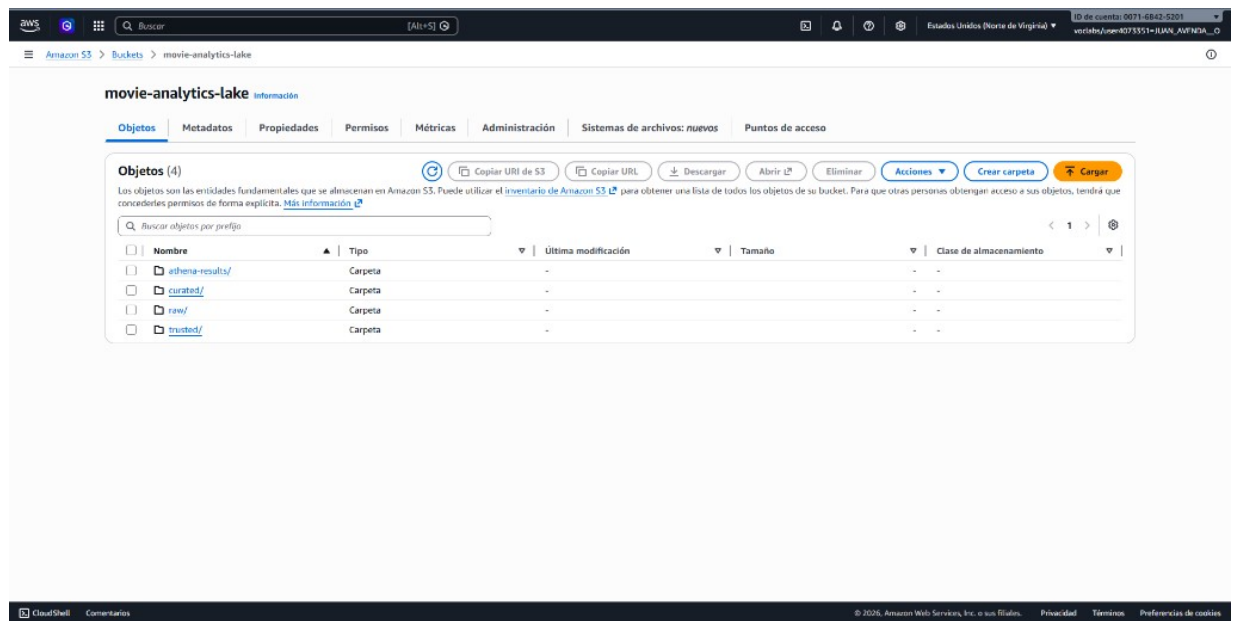


Figura 7: Zonas raw, trusted, curated y athena-results del bucket.

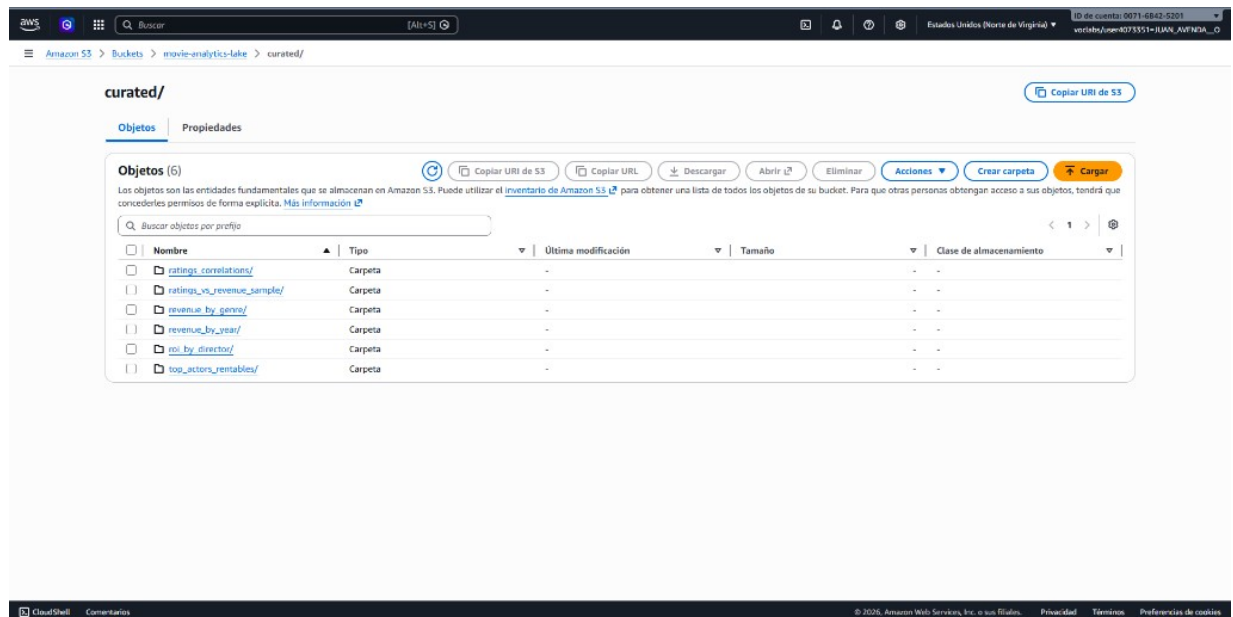


Figura 8: Las 6 tablas Parquet materializadas en curated/.

3.8. Punto 8 — Dashboard Streamlit + API Gateway

Objetivo

Exponer los resultados del análisis en un dashboard interactivo Streamlit desplegado en EC2 y publicado vía AWS API Gateway.

Dashboard desarrollado en **Streamlit** que lee la zona `curated/` con `s3fs` + `pyarrow` y presenta 4 KPIs globales y una visualización (Plotly) por cada pregunta de negocio.

KPIs del header: películas analizadas, revenue total USD (Billones), revenue total COP (Trillones), ROI promedio ponderado.

5 vistas:

- *Géneros*: barra horizontal top 15 (USD/COP) — `revenue.by_genre`.
- *Actores*: barra horizontal top 25 + tabla — `top_actors_rentables`.
- *Histórico*: líneas duales USD/COP por año — `revenue.by_year`.
- *Directores*: barra horizontal top 20 + tabla — `roi.by_director`.
- *Correlaciones*: 6 cards de correlación + scatter — `ratings_correlations`, `ratings_vs_revenue_sample`.

Estructura modular del código (punto8/app/): `streamlit_app.py` (orquestador), `config.py`, `styles.py`, `data.py`, `components.py`, y `tabs/` con un módulo `render()` por pregunta.

```
1 import pandas as pd
2 import streamlit as st
3 from config import CURATED_PATH
4
5 @st.cache_data(ttl=3600)
6 def load_curated(table_name: str) -> pd.DataFrame:
7     return pd.read_parquet(f"{CURATED_PATH}/{table_name}")
```

Listing 7: Carga cacheada desde `curated/`.

Despliegue en AWS: EC2 (Ubuntu 26.04 LTS) exponiendo Streamlit en el puerto 8501, detrás de un API Gateway HTTP API (`streamlit-dashboard-api`).

Nota técnica — API Gateway y WebSockets

Streamlit requiere *WebSockets* (`wss://.../_stcore/stream`) para refrescar gráficos en vivo. AWS HTTP API Gateway **no soporta el upgrade de protocolo a WebSocket**; la primera respuesta HTTP pasa correctamente, pero el canal bidireccional posterior no se establece.

Decisión adoptada: Mantuvimos el HTTP API como punto de entrada teórico y usamos la IP pública de la instancia. directa de la EC2 para las demostraciones interactivas. Las alternativas que sí soportan WebSocket nativo son CloudFront o un Application Load Balancer.

Cómo ejecutar:

1. Lanzar EC2 Ubuntu con LabRole y SG abriendo 22 (SSH) y 8501 (TCP).
2. Subir el código: `scp -i clave.pem -r punto8/app ubuntu@<EC2>:~/app`.
3. En la EC2: `python3 -m venv .venv \&\& source .venv/bin/activate`
4. Instalar las librerías necesarias para la app: `pip install -r app/requirements.txt`.
5. Lanzar: `streamlit run app/streamlit.app.py`
6. Crear API Gateway HTTP API con ruta ANY `/ {proxy+}` apuntando como integración HTTP a `http://<EC2_IP>:8501/{proxy}`.



Figura 9: Dashboard funcionando con datos reales (9.516 películas).

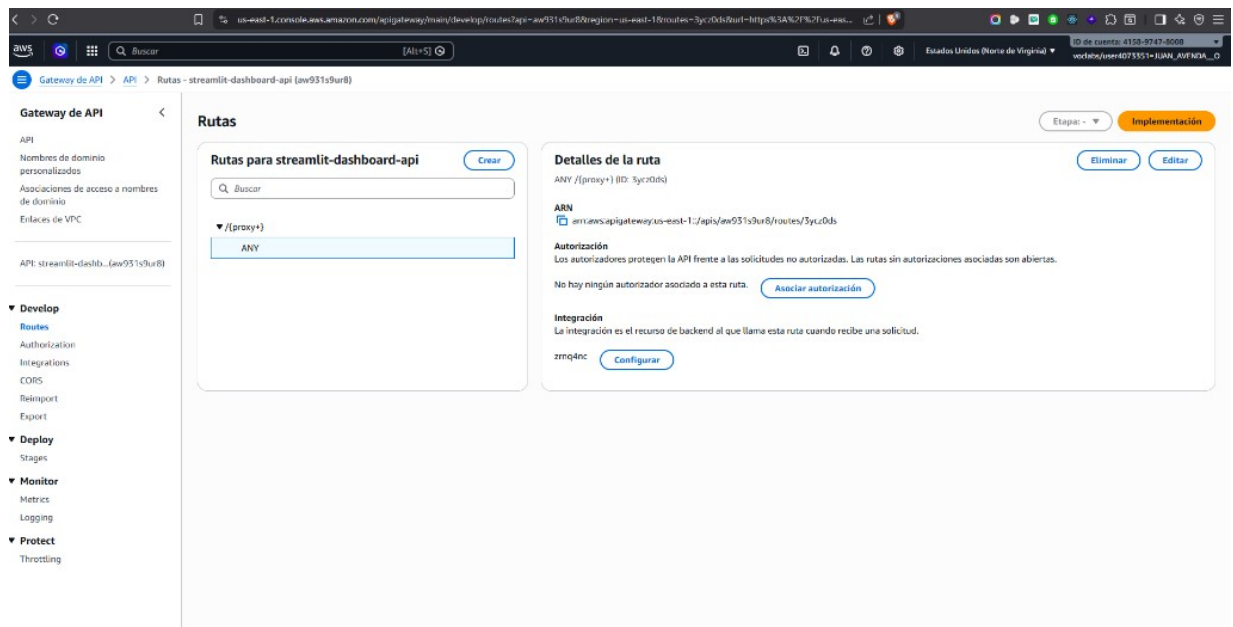


Figura 10: API Gateway HTTP API `streamlit-dashboard-api` con ruta `ANY /{proxy+}`.

4. Video de sustentación

La explicación del proyecto, en donde recorremos los 8 puntos sobre la consola de AWS y el dashboard final, se puede encontrar en:

drive.google.com/file/d/1D01IppYtNwM0kz4dhGvd3RCrR1eKDxP2